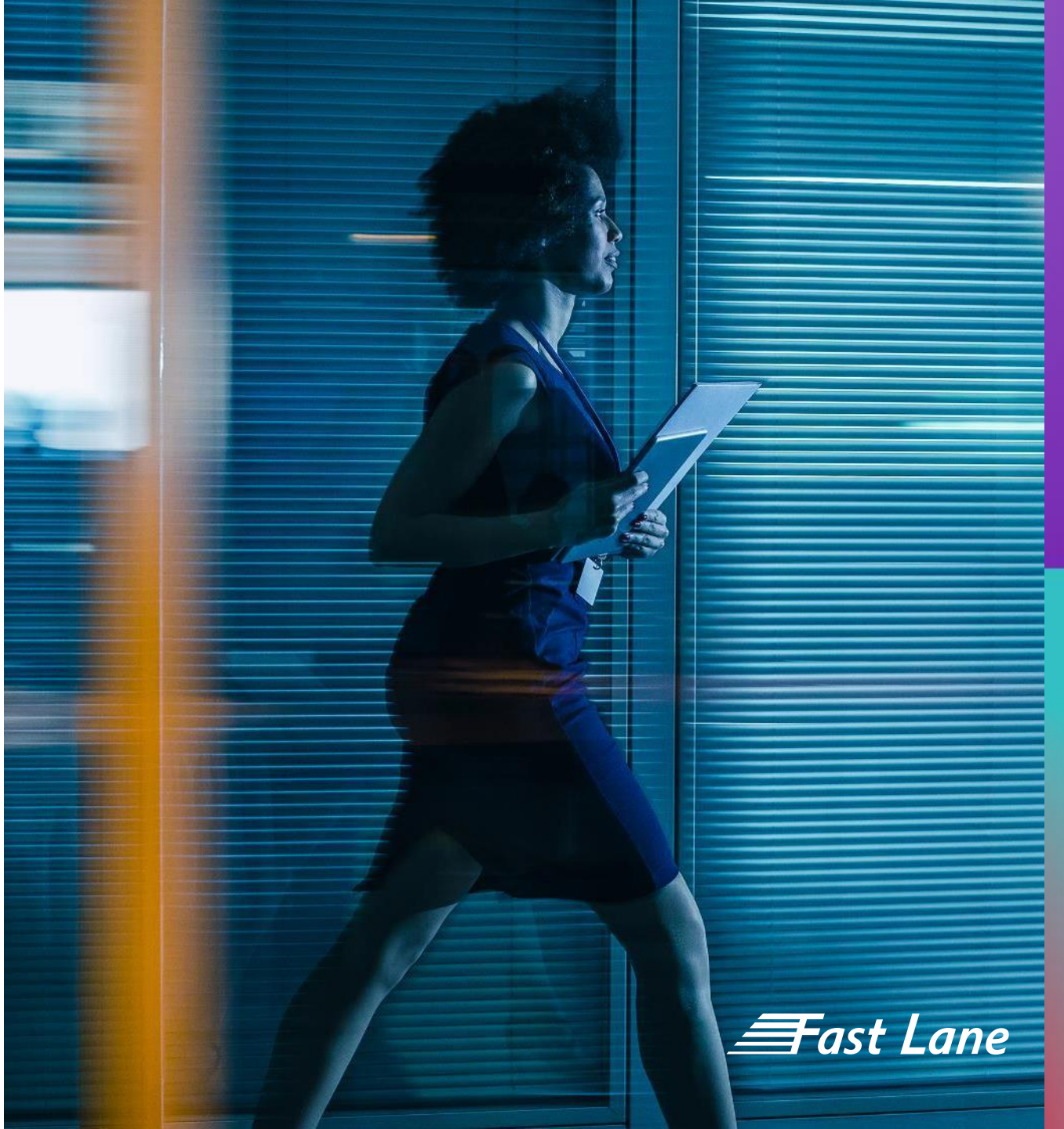
Fast Lane Group

Worldwide Education & Professional Services

Microsoft Solutions Partner

Microsoft Cloud Training Services



www.flane.de | © 2026 Fast Lane Institute for Knowledge Transfer GmbH

— AGENTIC AI: FROM STRATEGY TO **PRODUCTION.**

Alexander Kastil

Full stack, from enterprise software to agentic AI. Since 1996

AI at Work
Value discovery, KPI mapping and roadmap

Enterprise AI
Governed multi-agent systems on Microsoft Copilot and Foundry

Agentic Software Engineering
Harness architecture for Claude Code and GitHub Copilot

Team Enablement
AI driven content production and delivery

integrations.at
alexander.kastil@integrations.at

github.com/alexander-kastil
linkedin.com/in/alexander-kastil

INTEGRATIONS

AGENTIC AI CONSULTING · VIENNA



Microsoft CERTIFIED — AGENTIC AI BUSINESS SOLUTIONS ARCHITECT — ★★★

Microsoft CERTIFIED — AZURE AI APP AND AGENT DEVELOPER — ASSOCIATE ★★

Microsoft CERTIFIED — DEVOPS ENGINEER — EXPERT ★★★

SCAN TO CONNECT
integrations.at

MODULE 10

Build and host agents with the Agent SDK and Managed Agents

The agent loop in your code and in Anthropic's cloud



01

Core API Surface

query(), sessions, custom tools and typed output

02

Parallel Agents

Fan out, prove isolation, orchestrate

03

Managed Agents Foundations

Agent, Environment, Session and the state machine

04

Sessions & Streaming

Cloud containers, SSE events and steering

05

Production Features

Vaults, webhooks, policies and versions

06

Tools & MCP

MCP servers and skills on the Agent

07

Human-in-the-Loop

Approval gates and the migration path

08

Multiagent, Outcomes & Memory

Coordinated agents, graded results, memory stores

Agent SDK & Managed Agents

The agent loop in your code and in Anthropic's cloud

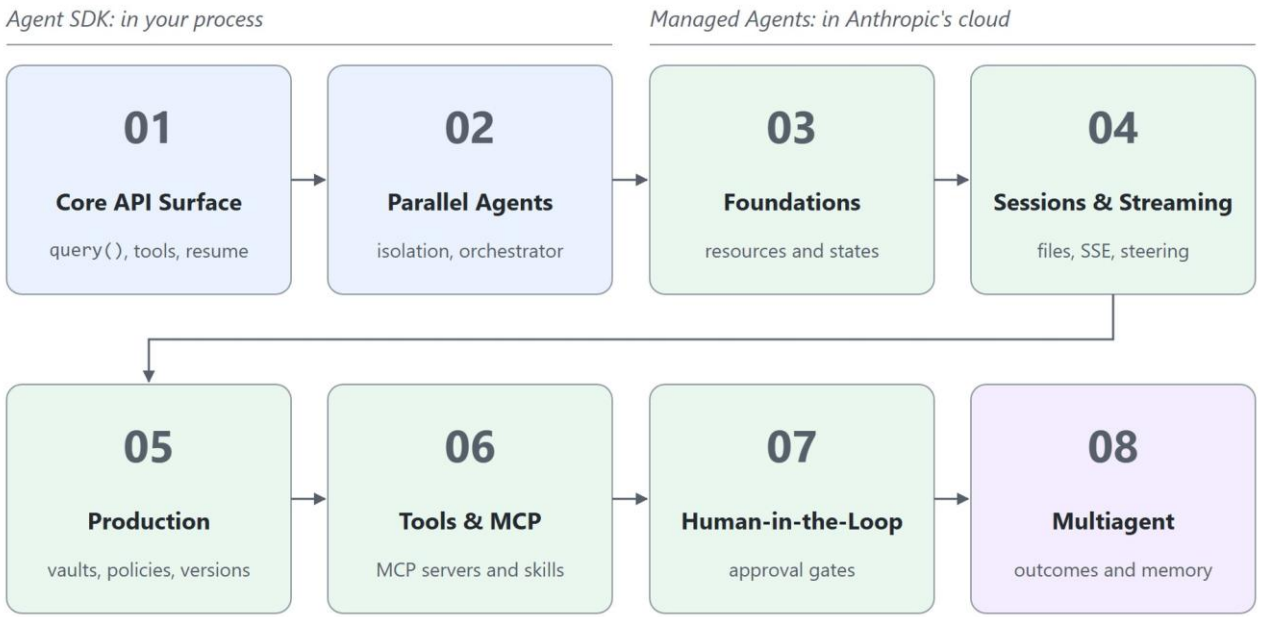
- **The loop as a library**: `query()` runs the Claude Code agent loop inside your program
- **Parallel agents**: isolated sessions fanned out with `Promise.all`, joined by an orchestrator
- **Hosted runtime**: Agent, Environment and Session run the loop in Anthropic's cloud



Overview

The path through the Agent SDK and Managed Agents

- Place the SDK between the **Messages API** and **Managed Agents**
- Call `query()`, add custom tools, resume a session, validate the result
- Fan out isolated sessions and join them with an **orchestrator**
- Hand the loop to the cloud: **Agent**, **Environment** and **Session**, streamed over **SSE**
- Go to production with vaults, **MCP**, skills, approval gates and **multiagent**



01

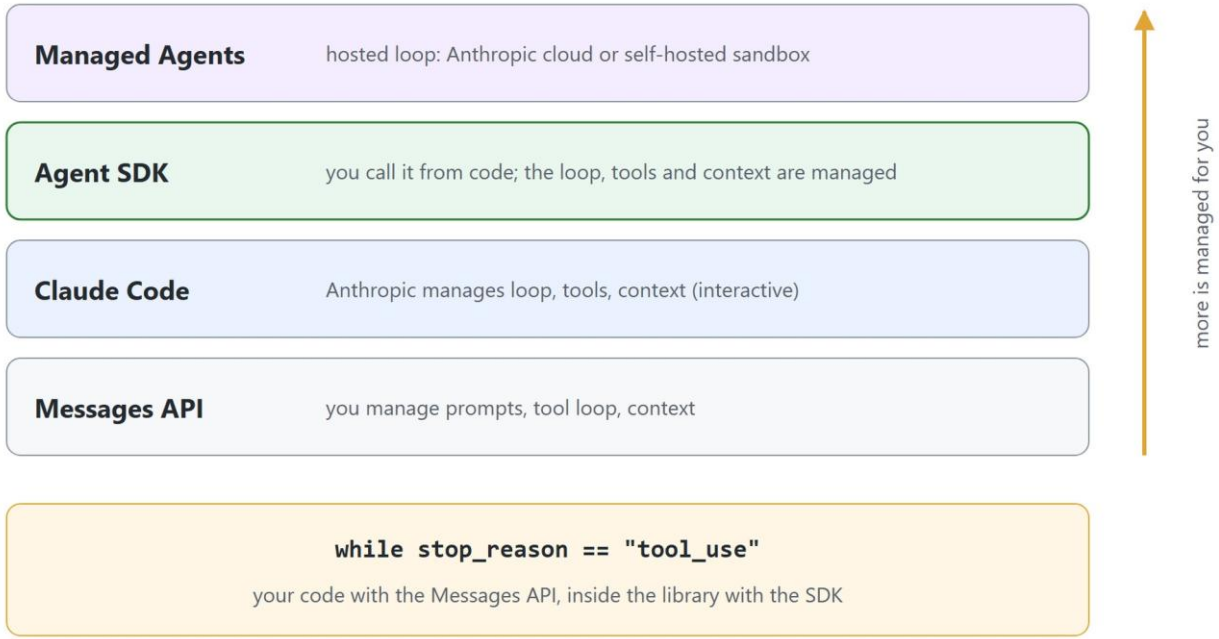
Core API Surface

query(), sessions, custom tools and typed output

Where the Agent SDK sits

Who runs the tool loop

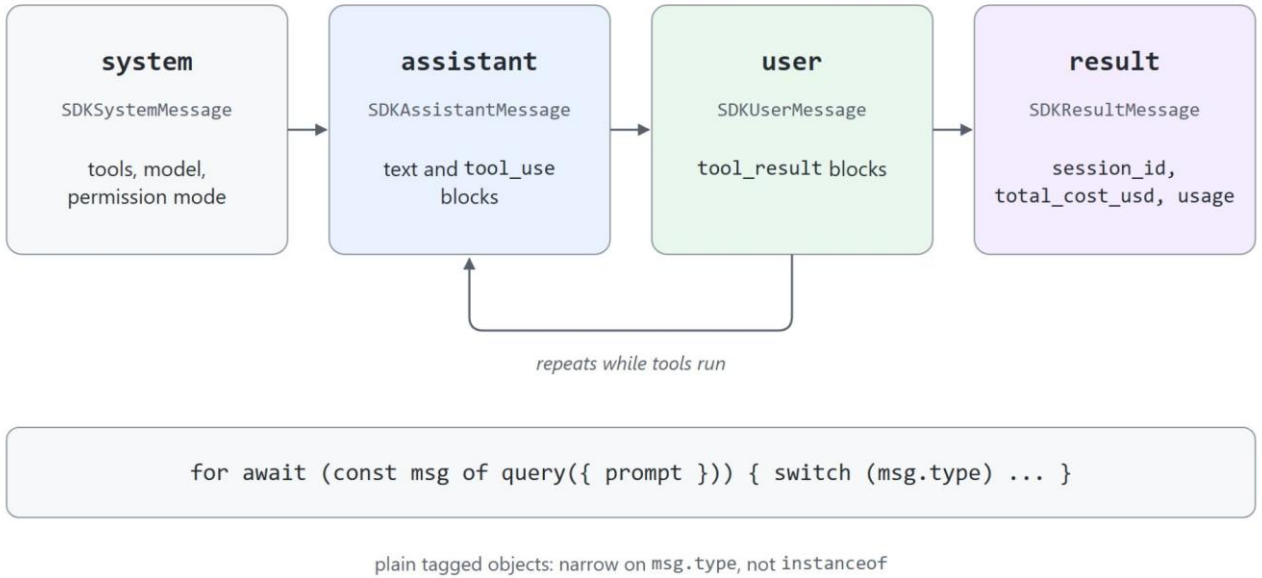
- **Messages API**: raw request and response, you write the tool loop
- **Agent SDK**: you call it from code, Claude runs the loop, you own the deployment
- **Managed Agents**: Anthropic hosts the loop, in its cloud sandbox or a self-hosted one
- The difference is who implements `while stop_reason == "tool_use"`



The query() async iterator

One event per turn of the agent loop

- `query()` is the single entry point and returns an **async iterator**
- Four message types: `system`, `assistant`, `user`, `result`
- Narrow on **`msg.type`**: messages are plain tagged objects, not classes
- Tool calls and results are content blocks in `msg.message.content`
- The `result` event carries `session_id`, cost and token usage



Choosing the SDK or the Messages API

Built-in tools against full control

- **Agent SDK** for built-in tools: file operations, bash, web search
- **Agent SDK** to embed Claude in CI, scripts and backend services
- **Agent SDK** to run several agents in parallel from one program
- **Messages API** for single-turn chat, full control of every turn, streaming chat UIs and tight latency



Options on every query() call

Allowed tools, permission mode, system prompt

- **`allowedTools`**: an allowlist of built-in tool names
- **`permissionMode`**: `default`, `acceptEdits`, `plan` or `bypassPermissions`
- **`systemPrompt`**: a persona layered on top of the default Claude Code prompt
- `bypassPermissions` also needs `allowDangerouslySkipPermissions: true`: sandboxed containers only
- TypeScript needs `zod@^4` as a peer dependency, or the install fails with `ERESOLVE`

```
query({ prompt, options: {
  allowedTools: ["Read", "Edit", "Bash"]
  permissionMode: "acceptEdits"
  systemPrompt: "You are a senior ..."
} })
```

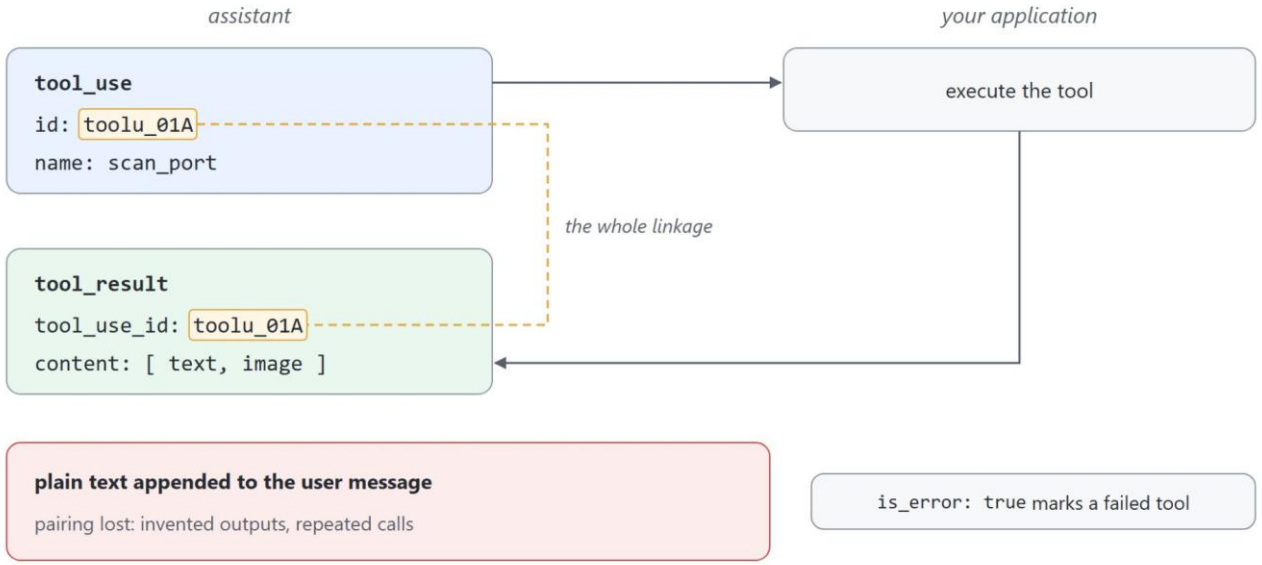
Mode	Behavior
`default`	prompts before dangerous operations
`acceptEdits`	auto-approves file edits
`plan`	plans only, executes no tool
`bypassPermissions`	allows everything, also needs `allowDangerouslySkipPermissions`

```
npm install @anthropic-ai/claude-agent-sdk zod@^4
```

The tool loop beneath query()

tool_use and tool_result joined by one id

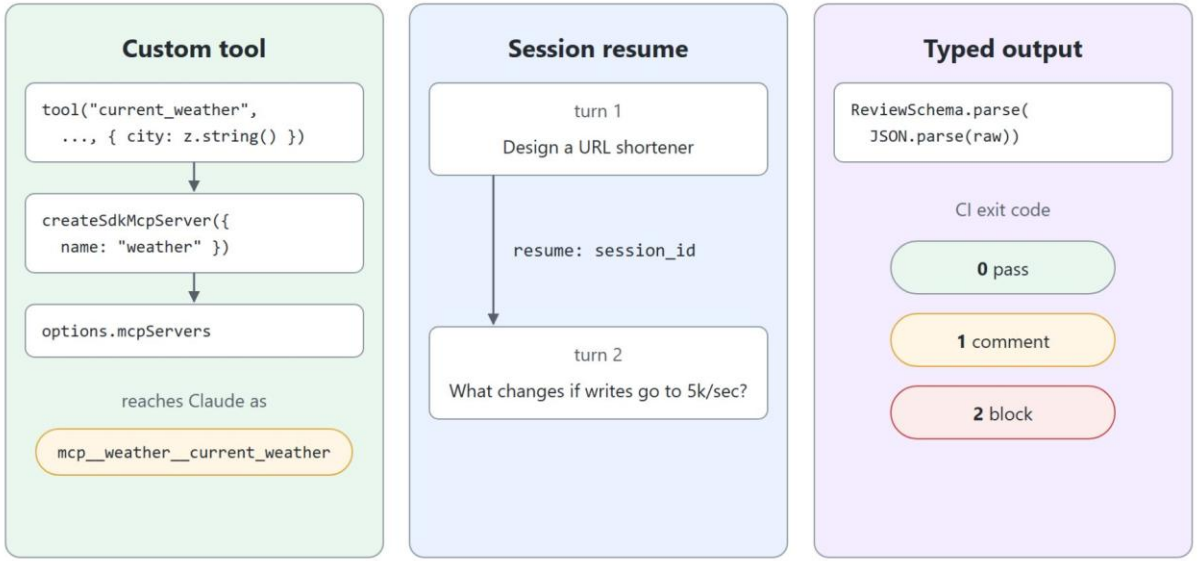
- The assistant emits a **`tool_use`** block with an id
- Your code answers with a **`tool_result`** whose `tool_use_id` matches
- That id is how the model maps an observation to its action
- `content` takes an array of blocks, so a tool can return an **image**
- Set **`is_error: true`** when the tool failed



Custom tools, session resume and typed output

From a weather tool to a CI exit code

- Declare a tool with **`tool()`** and a Zod shape, serve it with `createSdkMcpServer()`
- It reaches Claude as `mcp__<server>__<tool>`, the name you allowlist
- Pass a result's `session_id` as **`resume`** to continue with full context
- Validate JSON output with **Zod**: a malformed answer throws at the boundary
- The demo maps the verdict to exit codes 0, 1 and 2 for CI



Resume, fork or start fresh

Continuity for one line of work, isolation for a comparison

- **Resume**: yesterday's context is still valid, carry on
- **Fork**: two candidate solutions from one shared baseline
- **New session**: a task the old history would only pollute
- Reverting turns is no substitute for a fork
- A fork gets its own **session ID**; the filesystem stays shared

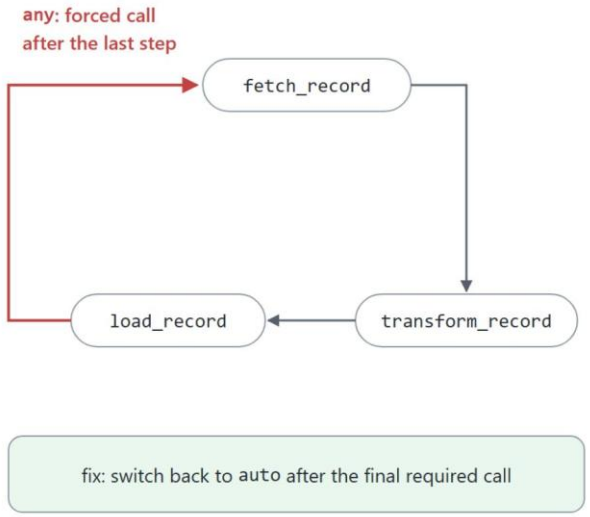


tool_choice

Forcing or releasing the tool call

- `auto`: the model decides, the default
- `any`: must call one of the tools; `tool`: must call exactly one
- `none`: may not call a tool
- `any` forces a call on **every** turn, so a finished pipeline loops
- Switch back to `auto` after the final required call

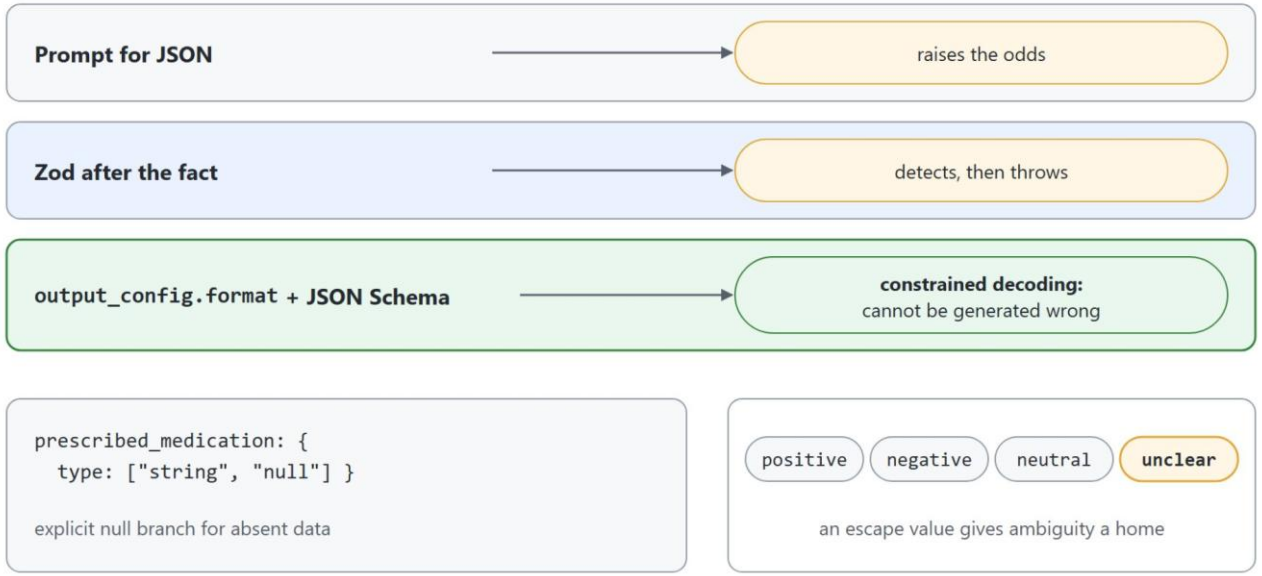
`auto`	model decides: tool or prose (default)
`any`	must call one of the tools
`tool, name: x`	must call exactly x
`none`	may not call a tool



Structured Outputs

A schema the response cannot violate

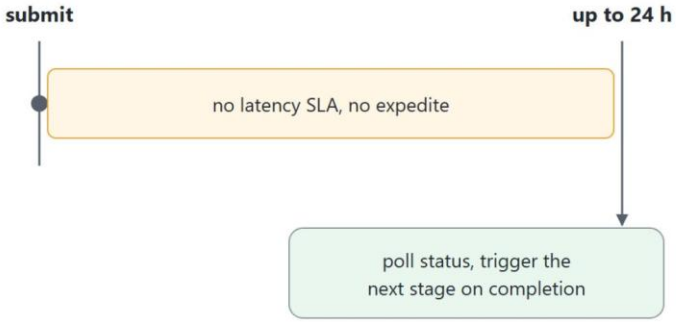
- A JSON Schema in **`output_config.format`** constrains decoding token by token
- Zod detects a bad answer; Structured Outputs **prevents** one
- Give absent data a legal value: `["string", "null"]`
- Every classifying enum needs an escape value such as **`unclear`**
- `output_format` is deprecated; use `output_config.format`



The Message Batches API

Half the cost, up to 24 hours

- Asynchronous processing at roughly **half the cost** of synchronous calls
- A completion window of **up to 24 hours**, no SLA, no expedite
- Right for overnight bulk work, wrong for anything a person waits on
- Poll the status and trigger the next stage on completion
- Refine the prompt on a **1,000-record sample** before the full run



Workload	Batch?
5 million legacy records overnight	yes
inline code completion in an IDE	no
nightly documentation regeneration	yes
support bot with a live user	no

refine the prompt on a 1,000-record sample with the synchronous API first

02

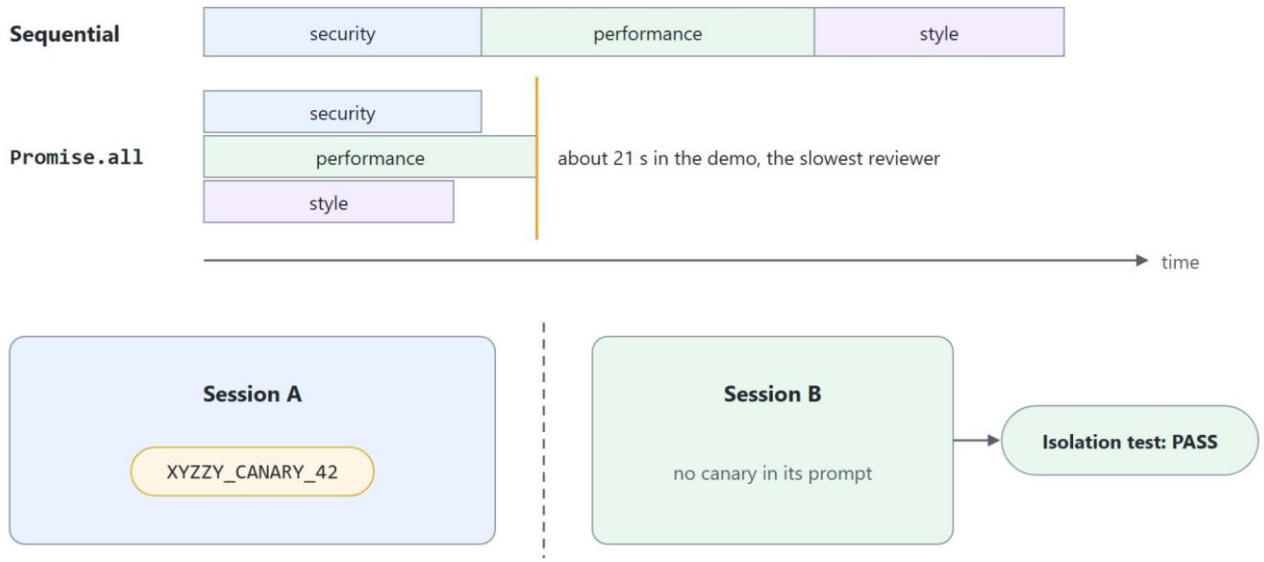
Parallel Agents

Fan out, prove isolation, orchestrate

Parallel sessions and isolation

Promise.all and the canary test

- `Promise.all` over several `query()` calls runs them **concurrently**
- Wall-clock time is the **slowest session**, not the sum
- Every call is a fresh session unless you pass `resume`
- The **canary test**: a word in session A must never appear in session B
- Run it once for every new parallel pipeline



The orchestrator and typed contracts

Three reviews in, one verdict out

- The orchestrator is a **fourth session** that reads only the reviewer outputs
- It deduplicates findings, resolves contradictions and issues **one verdict**
- Each reviewer returns `ReviewSchema`; the orchestrator returns `OrchestratorSchema`
- `approve` exits 0, `comment` exits 1, `block` exits 2



Three reviewers on one flawed file

Fixing flaws until the exit code moves

- Security, performance and style reviewers run over one deliberately flawed `auth.ts`
- Each surfaces its specialty: **SQL injection**, an unbounded cache, `db: any`
- The orchestrator lands on `block`: exit code 2
- Fixing only the SQL injection does not move the code: **every critical flaw** must go
- The full run spawns six sessions and takes roughly two minutes



03

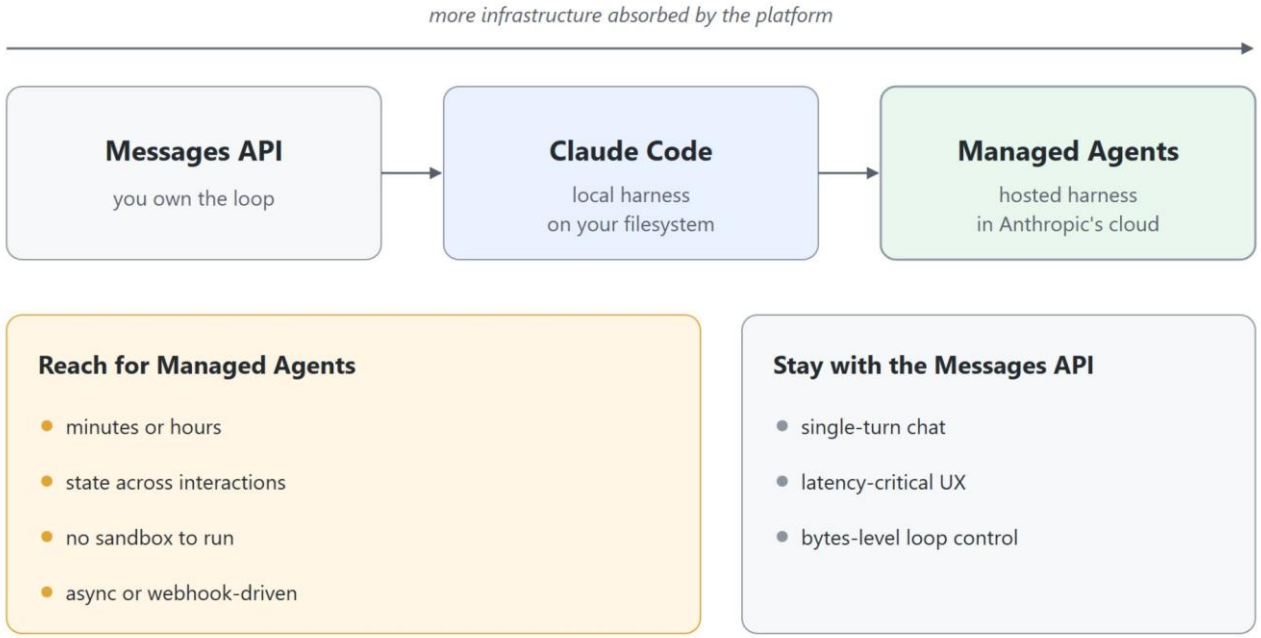
Managed Agents Foundations

Agent, Environment, Session and the state machine

Three layers of the agent loop

Messages API, Claude Code and Managed Agents

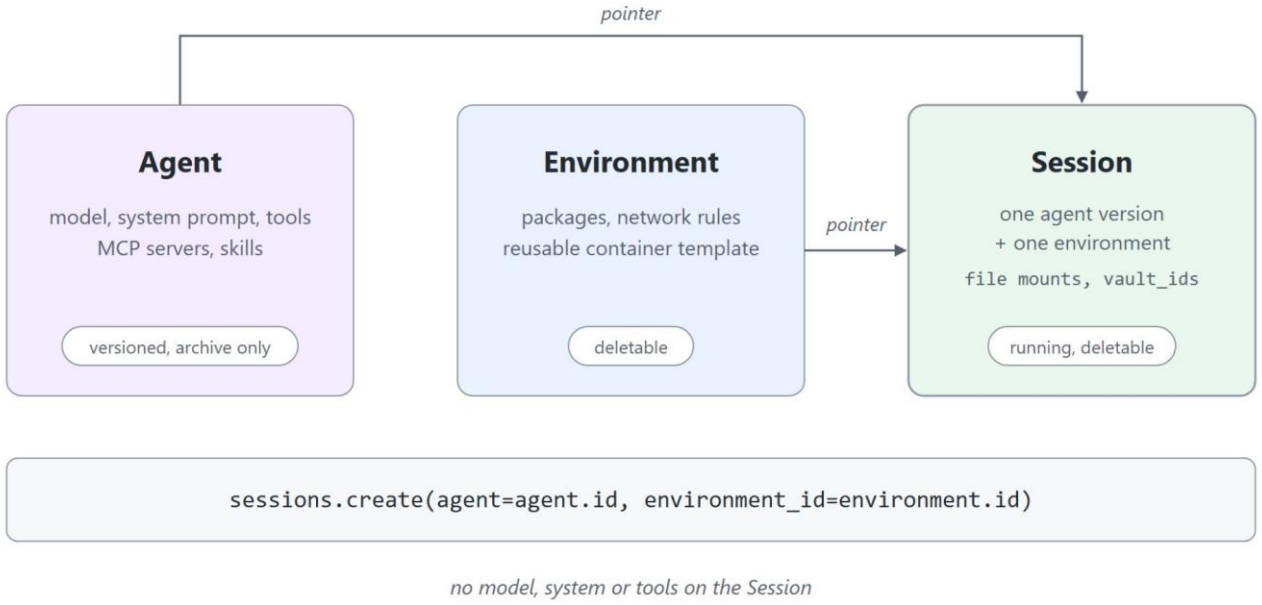
- **Messages API**: raw turns, you own the harness
- **Claude Code**: a local harness around your filesystem
- **Managed Agents**: the harness, container and state machine hosted behind one API
- Pays off for long, stateful, async or webhook-driven work
- Wrong choice for single-turn chat or latency-critical UX



Agent, Environment and Session

Two templates and the running thing

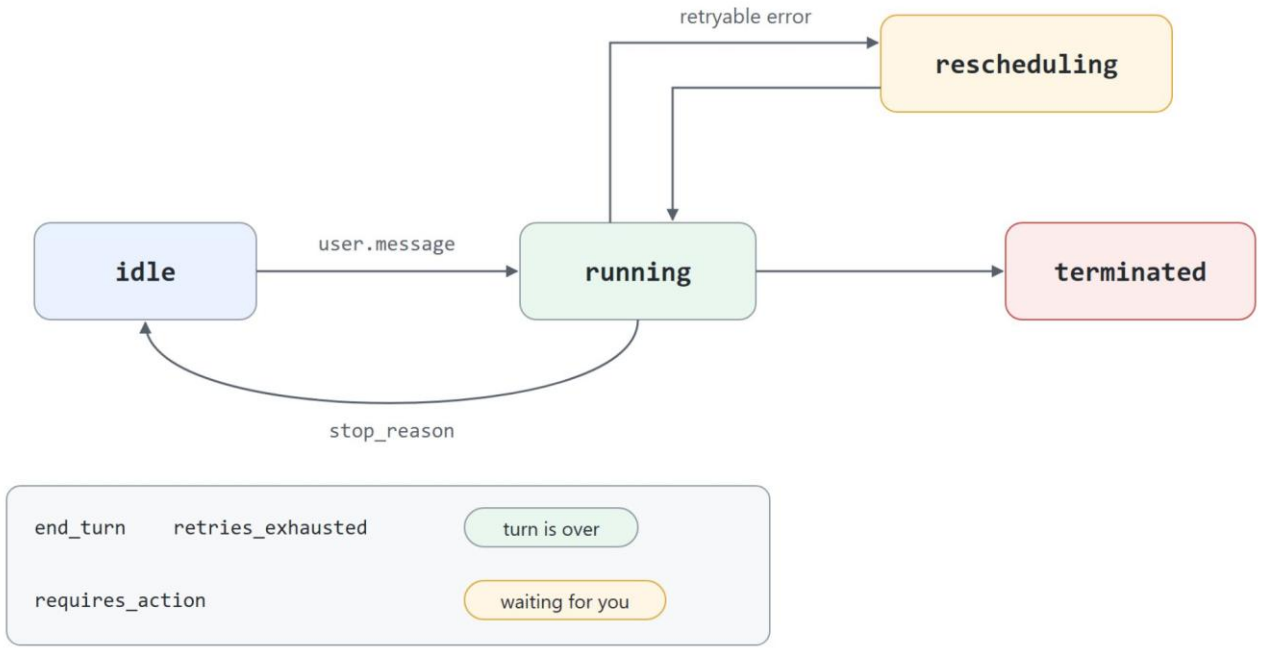
- **Agent**: model, system prompt, tools, MCP servers and skills, versioned
- **Environment**: the reusable cloud container definition
- **Session**: binds one agent version to one environment, plus mounts and vaults
- `model`, `system` and `tools` live on the Agent, never on the Session
- Sessions and Environments can be deleted; an Agent can only be archived



The session state machine

Idle, running, rescheduling and terminated

- A new Session starts at `idle`; a `user.message` makes it `running`
- It idles again on `end_turn`, `requires_action` or `retries_exhausted`
- `rescheduling` appears only while the platform retries a recoverable error
- `terminated` is irreversible
- Only `end_turn` and `retries_exhausted` mean the turn is over



04

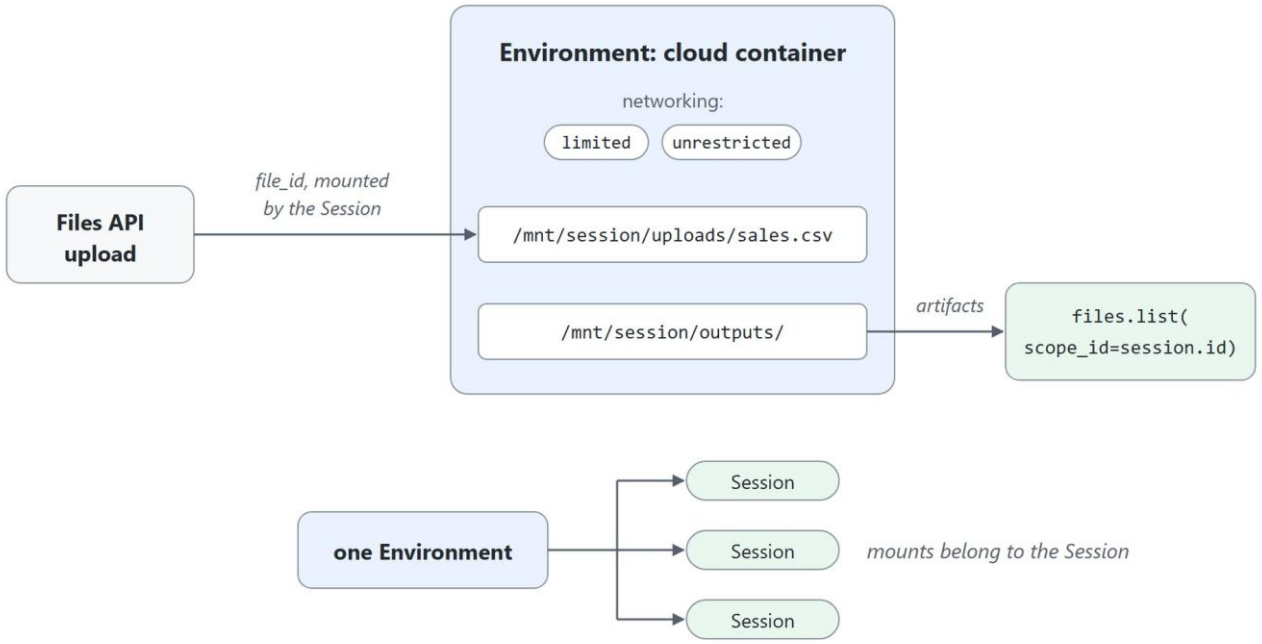
Sessions & Streaming

Cloud containers, SSE events and steering

Cloud containers and file mounts

Upload once, mount on the Session, read the outputs

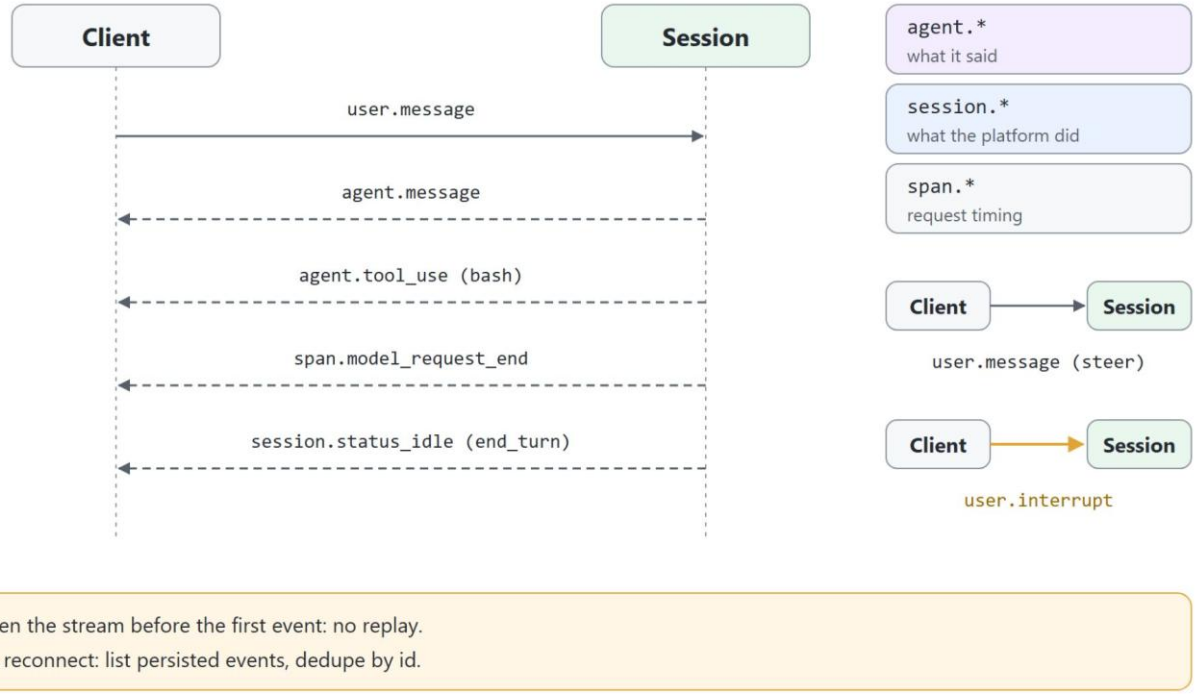
- An **Environment** is a cloud container Anthropic provisions, isolates and tears down
- Networking policy: `limited` or `unrestricted`
- Upload once, mount the file on the **Session** under `/mnt/session/`
- Mounts belong to the Session, so one Environment backs many runs
- Files written to `/mnt/session/outputs/` come back through the Files API



Events over SSE

The stream is the steering wheel

- Open an **SSE** stream, push `user.*` events in, read `agent.*`, `session.*` and `span.*` back
- Open the stream **before** the first event: it has no replay
- On reconnect, list persisted events and dedupe by event ID
- A new `user.message` mid-run steers the agent without cancelling
- `user.interrupt` jumps the queue and forces the Session to idle



05

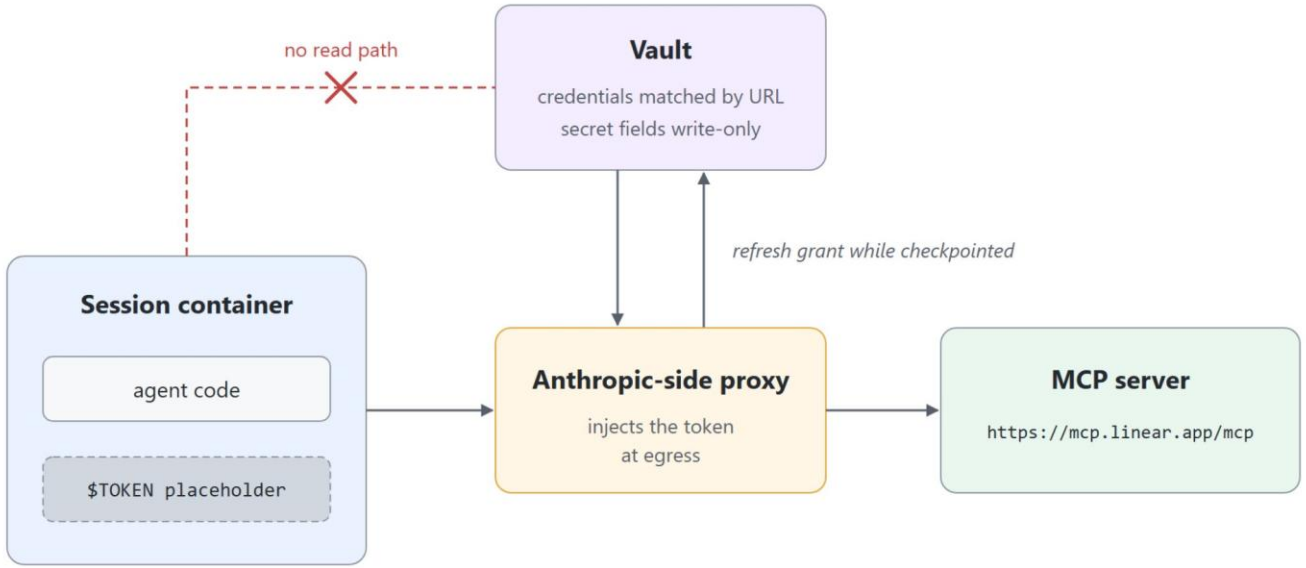
Production Features

Vaults, webhooks, policies and versions

Vaults and token refresh

Credentials the container never sees

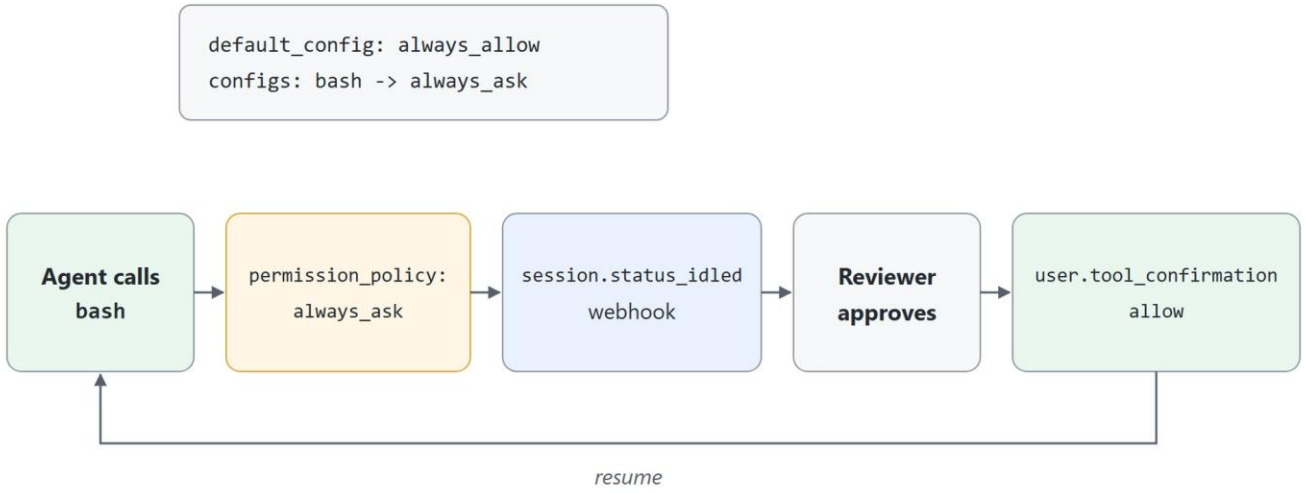
- A **vault** stores credentials the platform reads on the agent's behalf
- Attach with `vault_ids`; credentials match MCP servers by URL
- The `refresh` block lets Anthropic renew tokens while the Session sleeps
- Tokens are injected at egress by a proxy, never inside the container
- `environment_variable` credentials extend the same guarantee to CLIs



Permission policies and webhooks

Pause at the protected call, resume from your backend

- Policies attach **per tool**: `always_allow`, `always_ask` or `auto`
- Server-executed tools only; the agent toolset defaults to allow, MCP to ask
- A protected call parks the Session at `requires_action`
- A **webhook** on `session.status_idled` notifies your backend
- The backend resumes with `user.tool_confirmation` allow or deny

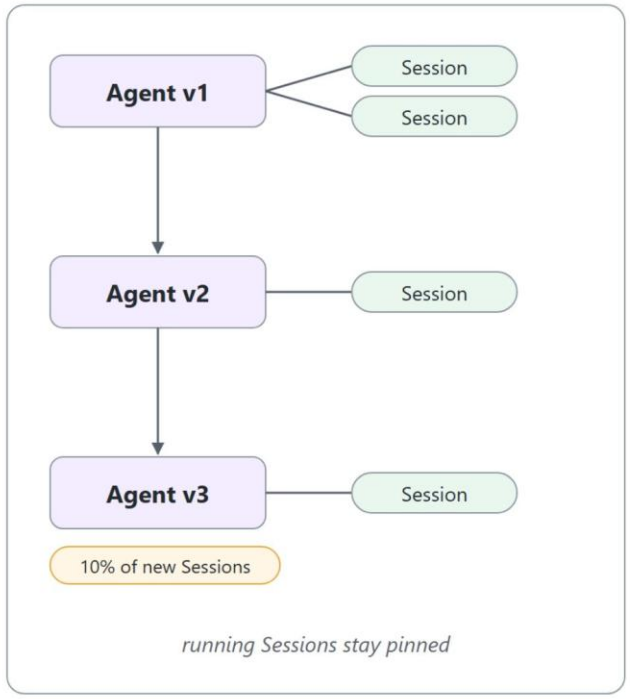


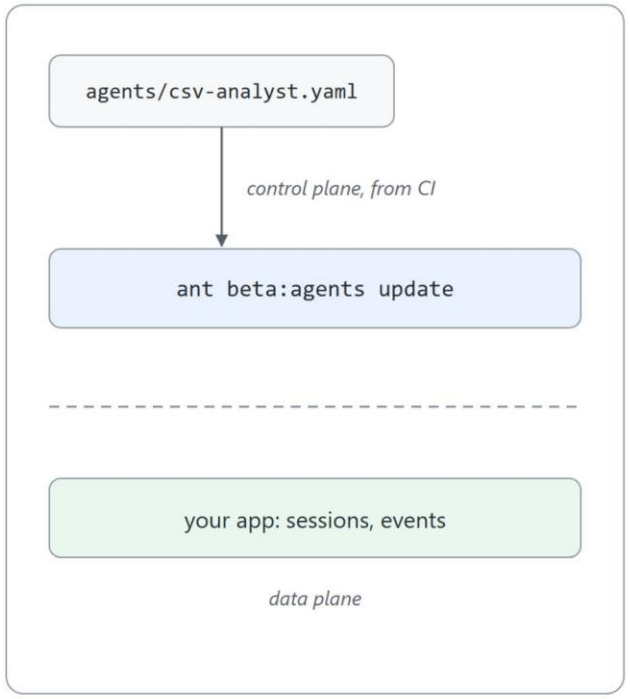
three policy types, for server-executed tools: built-in toolset and MCP

Agent versions and the ant CLI

Deployable units, kept in Git

- An Agent is immutable per version, like a container image tag
- Running Sessions stay pinned to the version they started on
- Staged rollout: route a share of new Sessions to the newest version
- `version` on `update` is an optimistic lock against racing operators
- The **`ant` CLI** applies Agent and Environment YAML from Git





06

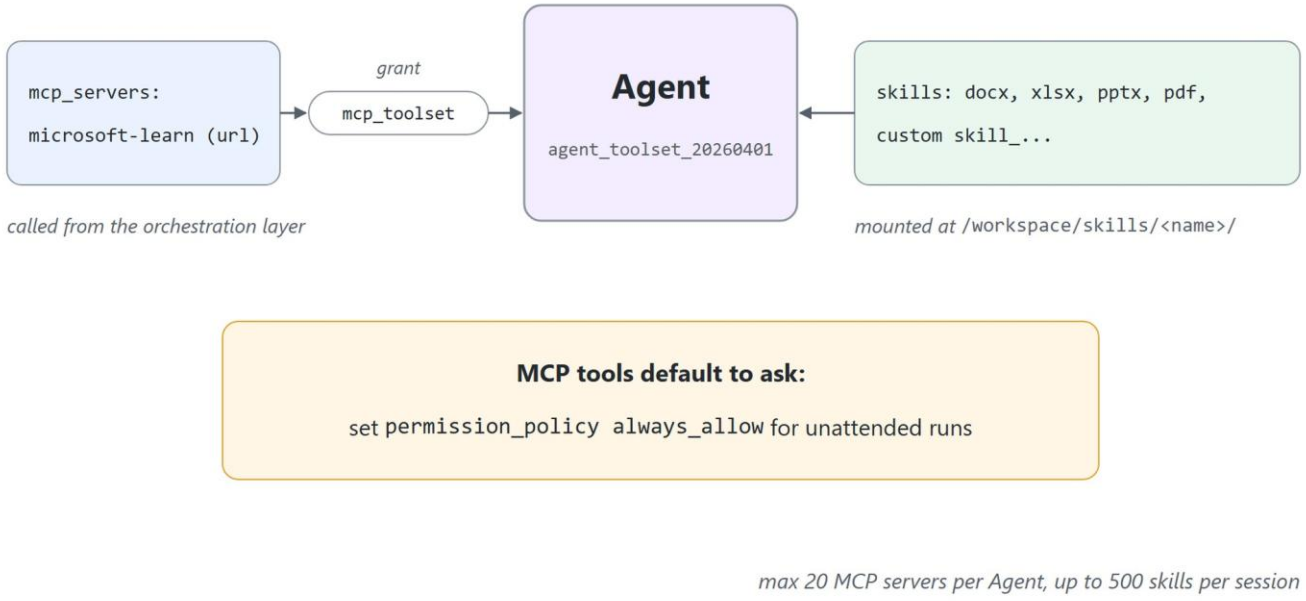
Tools & MCP

MCP servers and skills on the Agent

MCP servers and skills

Somewhere new to reach, or a documented way to do it

- `mcp_servers` declares a connection; an `mcp_toolset` entry grants it
- MCP tools run on the orchestration layer and default to **ask**
- **Skills** mount into the container and load their `SKILL.md` on demand
- Anthropic ships `docx`, `xlsx`, `pptx` and `pdf`; custom skills come from the Skills API
- A `limited` Environment needs `allow_mcp_servers` to reach the server



Reading the event log

Every MCP call and skill load is visible

- `agent.mcp_tool_use`: a tool on a connected MCP server ran
- `agent.tool_use`: a built-in tool ran in the container
- A skill load shows up as a `read` of `/workspace/skills/<name>/SKILL.md`
- Outputs in `/mnt/session/outputs/` are listed by session scope
- Indexing lags the idle event, so retry an empty list once



07

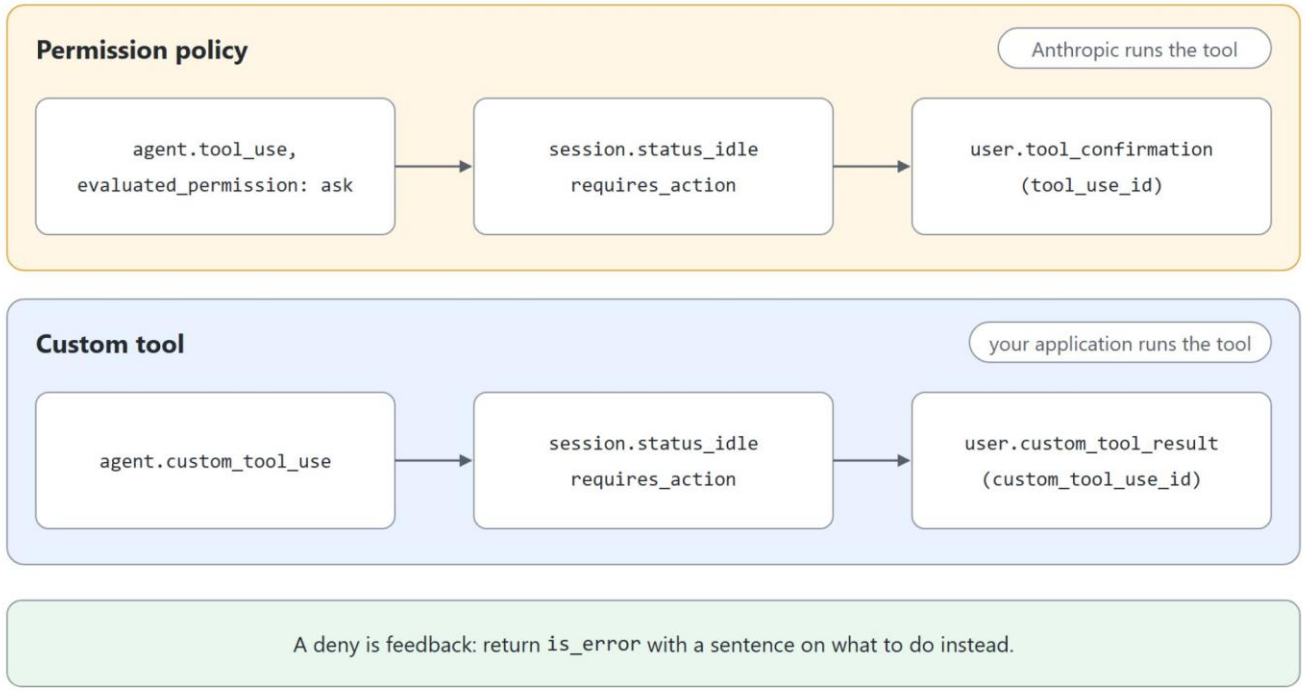
Human-in-the-Loop

Approval gates and the migration path

Two approval gates

Permission policy or custom tool

- **Permission policy**: Anthropic runs the tool, you allow or deny
- **Custom tool**: your application runs it and returns the result
- Both park the Session at `requires_action` with the pending `event_ids`
- Conflating `user.tool_confirmation` and `user.custom_tool_result` causes the 400s
- A deny with `is_error` and a reason lets the agent adapt



When the gate should fire

Observable triggers, not self-reported confidence

- The user explicitly asks for a human: honour it immediately
- Policy is silent and the stakes are material
- Repeated clarification has failed, or a lookup matches several real entities
- The required information is absent from the source: log, halt, flag
- Never escalate on a model's **self-reported confidence** score



08

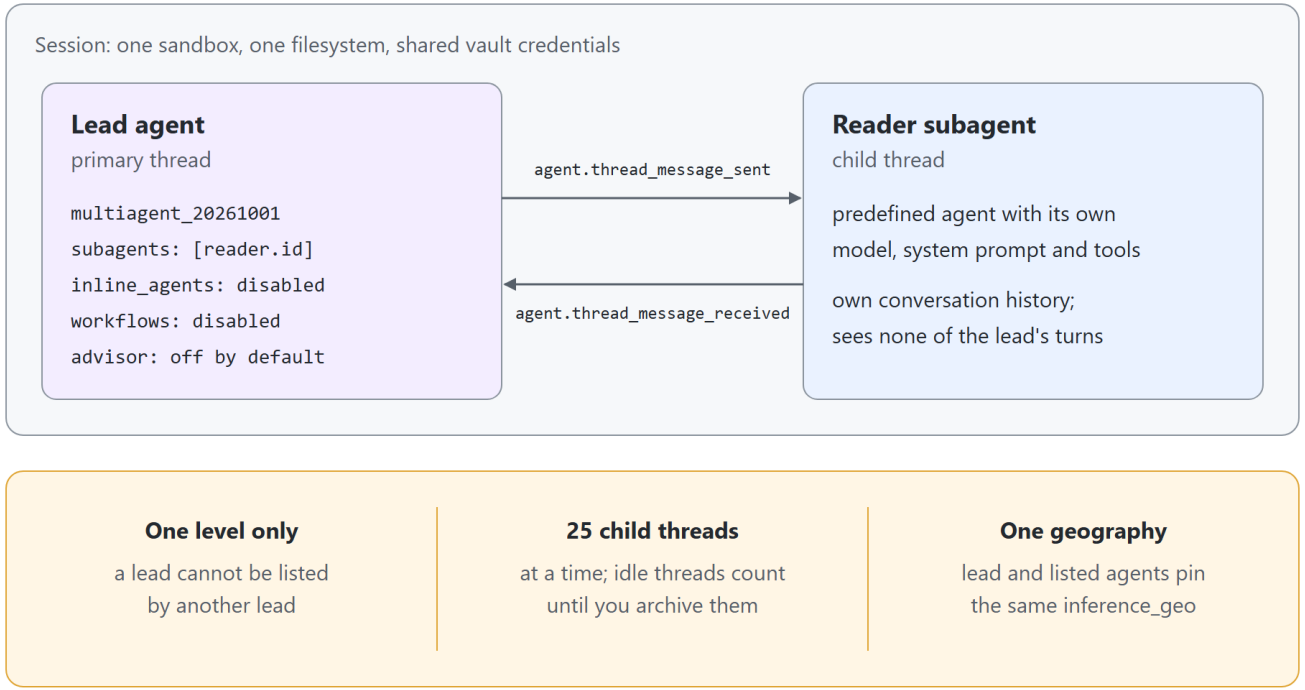
Multiagent, Outcomes & Memory

Coordinated agents, graded results, memory stores

The multiagent block

A lead that delegates to subagents in their own threads

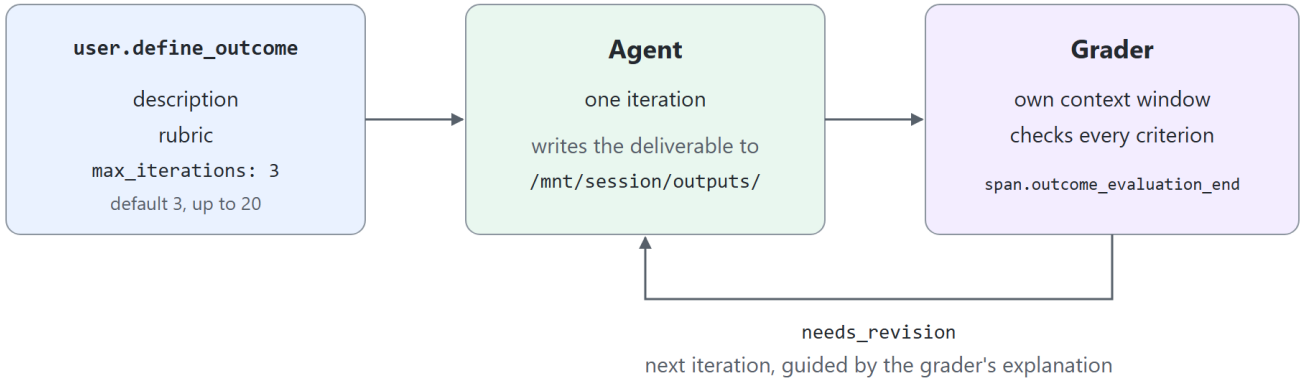
- `multiagent_20261001` sits on the Agent beside `system` and `tools`
- **subagents** and **workflows** list up to 20 agents; **advisor** is off by default
- `inline_agents` disabled keeps every task on the agents you chose
- One level only, and at most **25 child threads** at a time
- Threads share the sandbox and files; each keeps its own history



Outcomes and the grader

A rubric decides when the work is done

- `user.define_outcome` carries a description and a **rubric**, and starts the work
- A **grader** with its own context checks the deliverable against the rubric
- `needs_revision` starts another iteration; `max_iterations` defaults to 3, up to 20
- Write criteria the grader can check one at a time
- One outcome at a time; the deliverable lands in `/mnt/session/outputs/`

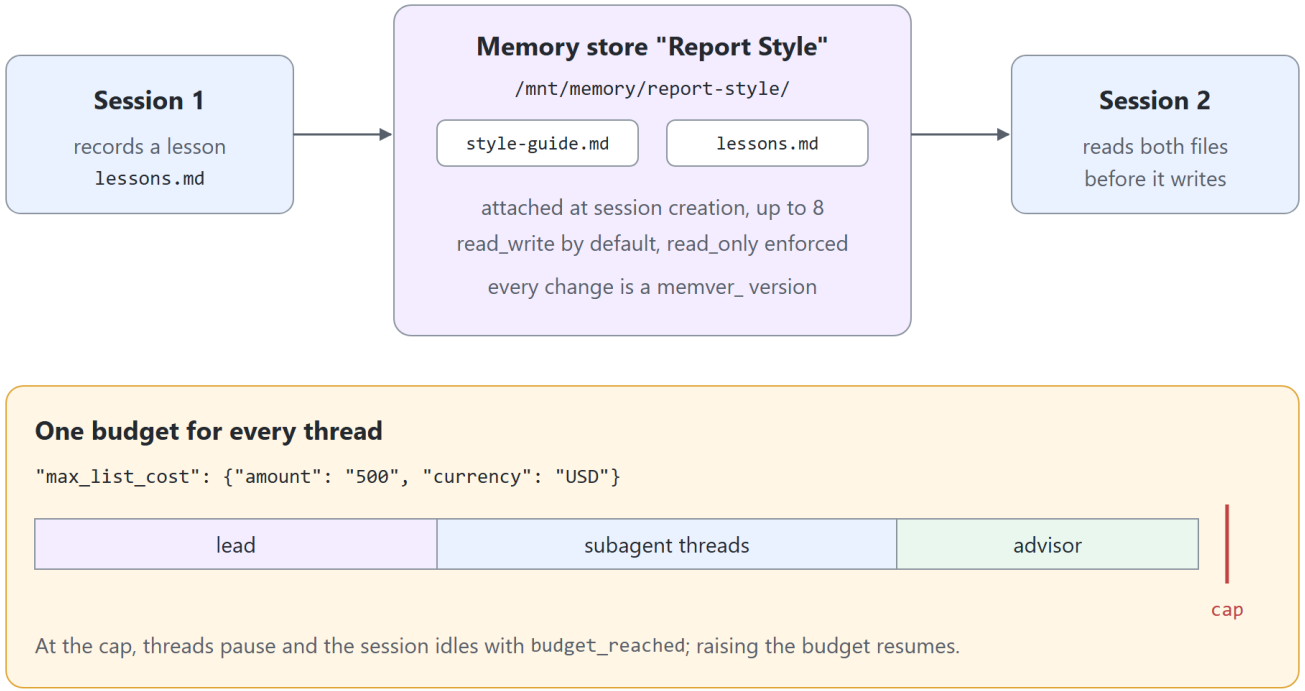


satisfied	the session goes idle
max_iterations_reached	one final acknowledgment turn, then idle
failed	idle: the rubric does not fit the deliverable
interrupted	a user.interrupt arrived during the outcome

Memory stores and the session budget

What outlives the session, and one cap for every thread

- A **memory store** holds small text documents that outlive any session
- Mounted under `/mnt/memory/<store>/`, read with the ordinary file tools
- Attach at session creation: up to 8 stores, `read_write` by default
- Attach reference material `read_only`; every write is a new version
- One **budget** caps every thread; at the cap the session idles with `budget_reached`



Agent SDK takeaways

What to carry into your own pipelines

- Reach for the **Agent SDK** when a program needs the Claude Code loop
- Keep every tool result linked to its call by `tool_use_id`
- Fan out with `Promise.all`, prove isolation with a **canary**
- Let an **orchestrator** reconcile, and map its verdict to an exit code
- Guarantee shapes with **Structured Outputs**, release `any` after the last step



Managed Agents takeaways

What to carry into production

- Graduate from the Messages API when work needs state, duration or approvals
- Version the Agent, reuse the Environment, mount files on the Session
- Open the stream first; break only on `end_turn` or `retries_exhausted`
- Keep credentials in **vaults** and gate risky tools with policies
- Escalate on observable triggers, never on self-reported confidence

